

**CENTRO UNIVERSITÁRIO DE MARINGÁ — UNIMA
CURSO DE GRADUAÇÃO EM ENGENHARIA DE SOFTWARE**

GUILHERME ROMUALDO

**MOTOR RAG (RETRIEVAL-AUGMENTED GENERATION) LOCAL:
PROJETO E IMPLEMENTAÇÃO MANUAL DE ESTRUTURAS DE
DADOS PARA BUSCA TEXTUAL OFFLINE**

MARINGÁ

2026

GUILHERME ROMUALDO

**MOTOR RAG (RETRIEVAL-AUGMENTED GENERATION) LOCAL:
PROJETO E IMPLEMENTAÇÃO MANUAL DE ESTRUTURAS DE
DADOS PARA BUSCA TEXTUAL OFFLINE**

Trabalho apresentado como requisito parcial à
conclusão da disciplina de Estrutura de Dados, do Curso
de Engenharia de Software do Centro Universitário de
Maringá — UNIMA.

MARINGÁ

2026

RESUMO

Este trabalho apresenta o projeto e a implementação de um Motor RAG (Retrieval-Augmented Generation) local, concebido para responder a consultas textuais sobre um corpus de documentos sem dependência de serviços externos. O sistema integra quatro estruturas de dados implementadas integralmente do zero pelo autor: uma Tabela Hash com tratamento de colisões por encadeamento separado, uma Árvore Trie para autocompletar via busca em profundidade (DFS), uma Árvore Splay para cache adaptativo de metadados e o algoritmo HeapSort com seleção Top-K para ordenação dos documentos por relevância. A pontuação dos resultados utiliza TF-IDF clássico calculado sobre o índice invertido construído a partir da tabela hash. O motor opera via linha de comando, processando um arquivo JSON de entrada e produzindo um arquivo JSON de saída determinístico. Três cenários de teste foram automatizados — básico, avançado e estresse — sendo este último composto por 10.000 documentos contendo 59.967 termos únicos e 500 consultas, atendendo e ultrapassando a meta de volume estabelecida. Os resultados demonstram tempo médio de 7,98 ms por consulta e pico de uso de memória de 195 MiB no cenário de estresse, com determinismo criptograficamente verificado entre execuções sucessivas.

Palavras-chave: estruturas de dados; tabela hash; árvore trie; árvore splay; heapsort; recuperação de informação.

SUMÁRIO

1 INTRODUÇÃO

2 FUNDAMENTAÇÃO TEÓRICA

2.1 Tabela Hash com Encadeamento Separado

2.2 Árvore Trie e Autocompletar por DFS

2.3 Árvore Splay e Cache Adaptativo

2.4 HeapSort e Seleção Top-K

2.5 Modelo de Pontuação TF-IDF

3 ARQUITETURA DO SISTEMA

4 IMPLEMENTAÇÃO DOS REQUISITOS FUNCIONAIS

4.1 RF01 — Índice Invertido sobre Tabela Hash

4.2 RF02 — Autocompletar via Trie e DFS

4.3 RF03 — Cache de Metadados via Splay Tree

4.4 RF04 — Ordenação Top-K via HeapSort

5 ANÁLISE DE COMPLEXIDADE

6 METODOLOGIA DE TESTES

7 RESULTADOS E PROVA DE CARGA

8 CONSIDERAÇÕES FINAIS

REFERÊNCIAS

1 INTRODUÇÃO

O paradigma Retrieval-Augmented Generation (RAG) tornou-se central em arquiteturas modernas de inteligência artificial aplicada a domínios fechados, nos quais a recuperação eficiente de fragmentos textuais previamente indexados precede a etapa de geração de respostas. Em cenários offline — sejam eles motivados por requisitos de confidencialidade, latência ou ausência de conectividade — o motor de recuperação deve operar de forma autossuficiente, encapsulando em memória todas as estruturas de indexação necessárias.

Este trabalho descreve o projeto, a implementação e a avaliação empírica de um Motor RAG local desenvolvido inteiramente em Python, no qual cada estrutura de dados central foi escrita do zero, sem recorrer a bibliotecas de alto nível ou contêineres prontos da linguagem hospedeira. O sistema atende a três operações principais: (i) busca textual com ranqueamento por relevância; (ii) autocompletar baseado em vocabulário indexado; e (iii) cache adaptativo de metadados frequentemente acessados.

A contribuição do trabalho é tripla. Primeiro, a entrega de uma implementação didática e reproduzível das quatro estruturas exigidas pelo enunciado — Tabela Hash, Trie, Árvore Splay e HeapSort. Segundo, a integração destas estruturas em uma arquitetura coesa que aplica o modelo TF-IDF clássico para pontuar documentos. Terceiro, a construção automatizada de um cenário de estresse compatível com a rubrica do projeto (10.000 documentos contendo no mínimo 50.000 termos únicos) acompanhado de um benchmark quantitativo de tempo de execução, uso de memória e determinismo da saída.

O texto está estruturado da seguinte forma. A Seção 2 apresenta a fundamentação teórica das estruturas e algoritmos utilizados. A Seção 3 descreve a arquitetura modular do sistema. A Seção 4 detalha a implementação de cada um dos quatro Requisitos Funcionais. A Seção 5 analisa formalmente a complexidade temporal e espacial das operações. A Seção 6 expõe a metodologia de testes adotada, incluindo geração automatizada de dados e verificação de determinismo. A Seção 7 reporta os resultados experimentais. A Seção 8 sintetiza as considerações finais.

2 FUNDAMENTAÇÃO TEÓRICA

2.1 Tabela Hash com Encadeamento Separado

Uma tabela hash é uma estrutura de dados que estabelece um mapeamento entre chaves e valores por meio de uma função de dispersão $h: K \rightarrow [0, m-1]$, onde m representa o tamanho do vetor de buckets. Quando duas chaves distintas k_1 e k_2 produzem o mesmo índice — situação denominada colisão — adota-se uma política de resolução. O encadeamento separado (separate chaining) armazena, em cada bucket, uma lista encadeada de pares (chave, valor) colidentes (CORMEN et al., 2009).

Sob a hipótese de hash uniforme simples — segundo a qual cada chave tem probabilidade $1/m$ de mapear-se a um bucket específico — as operações de inserção, busca e remoção apresentam custo esperado $\Theta(1 + \alpha)$, onde $\alpha = n/m$ denota o fator de carga. A estrutura implementada neste trabalho mantém $\alpha \leq 0,75$ através de redimensionamento automático (rehashing), evitando degradação assintótica das operações.

A função de dispersão adotada é uma variante do algoritmo DJB2 (BERNSTEIN, 1991), conhecida pela boa distribuição em chaves textuais. A escolha por uma função própria — e não pela primitiva `hash()` da linguagem hospedeira — garante a reprodutibilidade dos resultados independentemente do valor de `PYTHONHASHSEED`.

2.2 Árvore Trie e Autocompletar por DFS

A árvore de prefixos, ou Trie (FREDKIN, 1960), é uma estrutura arbórea ordenada na qual cada aresta é rotulada com um símbolo do alfabeto e cada caminho da raiz até um nó representa um prefixo do vocabulário indexado. Nós marcados como terminais correspondem a palavras completas. A grande vantagem da Trie sobre estruturas baseadas em comparação total de strings reside no fato de que o tempo de busca depende apenas do comprimento da chave consultada, e não do tamanho do dicionário.

Para a operação de autocompletar, descende-se até o nó correspondente ao prefixo informado e, a partir dele, executa-se uma busca em profundidade (DFS) que enumera todas as palavras armazenadas na subárvore. A DFS é tipicamente implementada de forma recursiva, contudo, para evitar limites de pilha em prefixos longos, adotou-se neste trabalho a variante iterativa baseada em uma estrutura de pilha explícita.

2.3 Árvore Splay e Cache Adaptativo

A Árvore Splay (SLEATOR; TARJAN, 1985) é uma árvore binária de busca auto-ajustável que se reorganiza a cada acesso, deslocando o elemento consultado até a raiz por meio de uma sequência de rotações conhecidas como operações zig, zig-zig e zig-zag. Embora não imponha invariantes estruturais de balanceamento — como faz a AVL ou a Red-Black —, a Splay assegura custo amortizado $O(\log n)$ para inserção, busca e remoção, e adicionalmente satisfaz o Teorema do Conjunto de Trabalho, segundo o qual sequências de acessos enviesadas (em que poucos elementos concentram a maior parte das consultas) tornam-se ainda mais eficientes que o logarítmico amortizado.

Estas propriedades fazem da Splay um candidato natural para implementar caches do tipo LRU (Least Recently Used), nos quais elementos quentes devem permanecer acessíveis com o menor número possível de operações.

2.4 HeapSort e Seleção Top-K

O HeapSort (WILLIAMS, 1964) é um algoritmo de ordenação por comparação que opera in-place em $O(n \log n)$ no pior caso, consistindo em duas fases: (i) construção de um heap binário a partir do vetor de entrada, em tempo linear; e (ii) extração sucessiva do elemento raiz, restabelecendo a propriedade de heap após cada extração, totalizando $O(n \log n)$ operações.

Para o problema correlato de selecionar os K maiores elementos de um conjunto de tamanho N ($K \ll N$), o emprego direto do HeapSort seria assintoticamente subótimo. A abordagem clássica consiste em manter uma min-heap de capacidade K , na qual a raiz armazena o menor dos K maiores correntes; quando um novo elemento supera essa raiz, ele a substitui e a propriedade de heap é restaurada. O custo total reduz-se a $O(N \log K)$, com uso de memória $O(K)$.

2.5 Modelo de Pontuação TF-IDF

O esquema TF-IDF (term frequency — inverse document frequency), introduzido por Salton (SALTON; BUCKLEY, 1988), é um método consagrado para atribuir pesos a termos em coleções textuais. A pontuação de um documento d em relação a uma consulta q é calculada conforme a expressão:

$$\begin{aligned} \text{score}(d, q) &= \sum_{t \in q} \text{tf}(t, d) \cdot \text{idf}(t) \\ \text{idf}(t) &= \log((1 + N) / (1 + \text{df}(t))) + 1 \\ \text{tf}(t, d) &= \text{count}(t, d) / \max(1, |d|) \end{aligned}$$

Nesta formulação, N denota o número total de documentos da coleção, $df(t)$ corresponde ao número de documentos que contêm o termo t e $|d|$ representa o comprimento (em tokens) do documento d . A suavização aditiva (+1) tanto no numerador quanto no denominador do idf evita divisões por zero e atenua o impacto de termos extremamente raros.

3 ARQUITETURA DO SISTEMA

O sistema foi desenvolvido em Python 3.11 e organiza-se em quatro camadas modulares: (i) estruturas de dados de baixo nível, localizadas em `src/structures`; (ii) camada de motor, em `src/engine`, responsável pela tokenização, scoring e orquestração; (iii) camada de geração de dados, em `src/generators`, encarregada de produzir massas de teste sintéticas; e (iv) camada de interface de linha de comando, em `src/main.py`, que recebe arquivos JSON de entrada e produz arquivos JSON de saída.

Nenhum dos arquivos da camada de estruturas importa bibliotecas externas ou contêineres prontos da biblioteca padrão (a exemplo de `collections.OrderedDict`, `heapq` ou `bisect`); a verificação por análise estática dos imports confirma o atendimento dessa restrição. A camada de geração de dados utiliza as bibliotecas `random` e `Faker`, ambas explicitamente permitidas pela rubrica como auxiliares à produção de dados falsos.

O fluxo de execução de uma consulta de busca é o seguinte. (1) O tokenizador normaliza a consulta aplicando a forma NFD do padrão Unicode e removendo diacríticos, garantindo que “árvore” e “arvore” sejam tratadas como o mesmo termo. (2) Para cada termo resultante, consulta-se o índice invertido — implementado sobre a Tabela Hash — obtendo a lista de documentos que contêm o termo, juntamente com a frequência. (3) Calcula-se o score TF-IDF acumulado por documento em uma tabela hash auxiliar. (4) Aplica-se a função `top_k`, baseada em min-heap, para selecionar os K documentos de maior pontuação em $O(N \log K)$. (5) Empates de pontuação são desempatados pela ordem lexicográfica do identificador do documento, garantindo determinismo da saída. (6) Cada documento retornado é registrado no cache Splay, sendo deslocado para a raiz da árvore.

4 IMPLEMENTAÇÃO DOS REQUISITOS FUNCIONAIS

4.1 RF01 — Índice Invertido sobre Tabela Hash

A classe `HashTable` implementa redimensionamento automático (rehashing) sempre que o fator de carga ultrapassa 0,75. A capacidade inicial é arredondada para a próxima

potência de dois, o que permite substituir a operação de módulo pela operação bit-a-bit AND com (capacidade - 1), reduzindo o custo da indexação.

Sobre essa estrutura, a classe `InvertedIndex` constrói um mapeamento termo $\rightarrow$ {doc_id $\rightarrow$ frequência}, no qual a posting list de cada termo é, por sua vez, uma `HashTable` de (doc_id $\rightarrow$ contagem). Esta composição mantém todas as operações de atualização e consulta do índice em $O(1)$ esperado.

4.2 RF02 — Autocompletar via Trie e DFS

Cada nó da Trie referencia seus filhos através de uma instância da `HashTable` própria — não do dicionário nativo da linguagem —, reforçando a coerência metodológica do projeto. A operação `autocomplete` localiza primeiro o nó correspondente ao prefixo consultado em $O(L)$, onde L é o comprimento do prefixo, e em seguida executa uma busca em profundidade iterativa que enumera as palavras da subárvore. Os filhos são processados em ordem lexicográfica para que a saída seja determinística e independente da ordem de inserção.

A camada superior reordena o conjunto de sugestões por frequência decrescente utilizando o `HeapSort` externo (RF04), de modo que as palavras mais comuns aparecem primeiro. Quando há empate de frequência, o desempate ocorre pela ordem lexicográfica ascendente.

4.3 RF03 — Cache de Metadados via Splay Tree

A árvore `Splay` é parametrizada por uma capacidade máxima (default = 256, configurada para 512 no cenário de estresse). Quando essa capacidade é excedida, dispara-se uma política de ejeção determinística que percorre a árvore a partir da raiz sempre pelo filho esquerdo (e, na ausência deste, pelo direito) até atingir uma folha, a qual é então removida. Esta heurística aproveita a propriedade fundamental da `Splay` — itens recentemente acessados situam-se próximos à raiz — para garantir que a folha mais distante represente um item frio do ponto de vista do padrão de acessos.

4.4 RF04 — Ordenação Top-K via HeapSort

Duas funções compõem este requisito. A função `heap_sort` executa a ordenação completa de um vetor in-place, com construção de heap em $O(n)$ seguida de $n-1$ extrações em $O(\log n)$ cada. A função `top_k`, por sua vez, mantém uma min-heap de tamanho K sobre o fluxo de entrada: para cada novo elemento, se a heap ainda não atingiu K elementos, insere-se; caso contrário, comparara-se com a raiz e substitui-se quando o novo elemento for maior.

Ao final, ordena-se a heap residual em ordem decrescente. O custo total é $O(N \log K)$, com memória adicional $O(K)$.

5 ANÁLISE DE COMPLEXIDADE

A Tabela 1 sintetiza as complexidades temporais e espaciais das operações principais. Adotam-se as seguintes convenções: N denota o número de documentos indexados; V o tamanho do vocabulário (termos únicos); L o comprimento de uma palavra ou prefixo; e K o limite de retorno da busca Top- K .

Tabela 1 — Complexidades das operações principais.

Operação	Caso médio	Pior caso	Memória
HashTable.put / get / remove	$O(1)$	$O(N)$	$O(N)$
InvertedIndex.add_document	$O(D)$	$O(D \cdot N)$	$O(V \cdot N)$
Trie.insert / contains	$O(L)$	$O(L)$	$O(L)$
Trie.autocomplete(prefix, K)	$O(L + S \log S)$	$O(L + V)$	$O(V)$
SplayTree.insert / get / remove	$O(\log N)^*$	$O(N)$	$O(N)$
heap_sort(arr)	$O(N \log N)$	$O(N \log N)$	$O(1)$
top_k(arr, K)	$O(N \log K)$	$O(N \log K)$	$O(K)$
search(query, K) — total	$O(q \cdot M + N \log K)$	—	$O(N)$

* Complexidade amortizada, conforme demonstrado por Sleator e Tarjan (1985). Na linha de search, M denota o tamanho médio da posting list dos termos da consulta.

6 METODOLOGIA DE TESTES

A estratégia de validação articula-se em três níveis complementares. Primeiro, testes unitários sobre cada estrutura de dados, escritos com a biblioteca unittest da biblioteca padrão do Python, cobrem inserção, busca, remoção, iteração e casos limítrofes. Foram especificados 18 casos de teste, incluindo um teste de estresse aleatório sobre a HashTable com 2.000 operações dirigidas por uma instância de random.Random com semente fixa.

Segundo, testes de aceitação executados sobre três cenários completos — denominados básico, avançado e estresse — cujas saídas reais são comparadas, item por item,

com gabaritos previamente armazenados no diretório data. A ferramenta `src.tools.diff_outputs` realiza essa comparação ignorando o campo estatísticas (que contém medições temporais dependentes do hardware) e validando estritamente o campo resultados.

Terceiro, um benchmark dedicado, encapsulado em `src.tools.stress_bench`, mede tempo de execução e pico de uso de memória residente (RSS) ao processar o cenário de estresse. A geração das massas de teste é integralmente automatizada por `src.generators.generate_data`, sendo controlada por uma semente fixa (`seed = 42`) que garante reprodutibilidade.

7 RESULTADOS E PROVA DE CARGA

Todos os experimentos foram conduzidos em um computador equipado com processador Apple Silicon e 16 GB de memória RAM, sob o sistema operacional macOS 14. A versão da linguagem hospedeira é Python 3.11.

Tabela 2 — Volume de dados dos três cenários de teste.

Cenário	Documentos	Termos únicos	Consultas	Tamanho do input
Básico	5	57	7	2,1 KiB
Avançado	8	82	11	3,2 KiB
Estresse	10.000	59.967	500	6,4 MiB

Tabela 3 — Medições no cenário de estresse.

Métrica	Valor obtido	Meta da rubrica
Documentos indexados	10.000	≥ 10.000
Termos únicos no vocabulário	59.967	≥ 50.000
Consultas processadas	500	—
Tempo de indexação	$\approx 5,3$ s	—
Tempo de 500 consultas	$\approx 4,0$ s	—
Tempo médio por consulta	$\approx 7,98$ ms	—
Tempo total de processo	$\approx 9,4$ s	—
Pico de memória residente (RSS)	≈ 195 MiB	sem estouro

Tabela 4 — Estatísticas internas das estruturas após o cenário de estresse.

Estrutura	Métrica	Valor
HashTable (índice)	tamanho	59.967
HashTable (índice)	capacidade	131.072
HashTable (índice)	fator de carga	0,4575

HashTable (índice)	cadeia máxima	5
HashTable (índice)	cadeia média não vazia	1,24
Trie	palavras	59.967
Trie	nós	242.903
Trie	profundidade máxima	14
SplayTree	tamanho	512
SplayTree	cache hits	280
SplayTree	cache misses	1.276
SplayTree	ejeções	764

Os resultados consolidados nas Tabelas 2 a 4 evidenciam o atendimento integral às metas da rubrica. Em particular, o vocabulário final do cenário de estresse supera em aproximadamente 19% o piso de 50.000 palavras únicas exigido. A cadeia máxima de 5 elementos na Tabela Hash, frente a uma população de aproximadamente 60.000 chaves, demonstra que a função de dispersão DJB2 oferece distribuição satisfatória para o domínio textual avaliado, confirmando o caso médio $O(1)$ das operações.

O determinismo da saída foi verificado de duas formas. Primeiramente, pela comparação posicional entre o gabarito e a saída real, conduzida pela ferramenta `diff_outputs`, com sucesso integral nos três cenários. Em seguida, pela inspeção do hash criptográfico SHA-256 do campo resultados entre execuções sucessivas do cenário de estresse, que produziram exatamente o mesmo dígito, fornecendo evidência forte da estabilidade da saída.

8 CONSIDERAÇÕES FINAIS

Este trabalho apresentou um Motor RAG local com quatro estruturas de dados implementadas manualmente — Tabela Hash, Trie, Árvore Splay e HeapSort — integradas em uma arquitetura modular capaz de processar consultas de busca, autocompletar e leitura de metadados com tempo médio inferior a 10 ms por operação em um corpus de 10.000 documentos. A automação completa da geração de massas de teste, somada à verificação de determinismo via comparação criptográfica, confere ao projeto a reprodutibilidade necessária para auditorias acadêmicas.

Como trabalhos futuros sugerem-se: (i) a substituição do ranqueamento TF-IDF por BM25 (ROBERTSON; ZARAGOZA, 2009), tipicamente mais robusto para coleções com documentos de comprimentos heterogêneos; (ii) a serialização do índice invertido em disco

mediante segmentação à la Lucene, permitindo que o corpus exceda a memória disponível; e (iii) a complementação do índice textual por embeddings vetoriais computados offline, evoluindo o motor para um sistema híbrido léxico-semântico.

REFERÊNCIAS

- BERNSTEIN, D. J. djb2: a hash function for strings. Usenet post comp.lang.c, 1991.
- CORMEN, T. H.; LEISERSON, C. E.; RIVEST, R. L.; STEIN, C. Introduction to Algorithms. 3. ed. Cambridge: MIT Press, 2009.
- FREDKIN, E. Trie memory. Communications of the ACM, v. 3, n. 9, p. 490–499, 1960.
- ROBERTSON, S.; ZARAGOZA, H. The probabilistic relevance framework: BM25 and beyond. Foundations and Trends in Information Retrieval, v. 3, n. 4, p. 333–389, 2009.
- SALTON, G.; BUCKLEY, C. Term-weighting approaches in automatic text retrieval. Information Processing & Management, v. 24, n. 5, p. 513–523, 1988.
- SLEATOR, D. D.; TARJAN, R. E. Self-adjusting binary search trees. Journal of the ACM, v. 32, n. 3, p. 652–686, 1985.
- WILLIAMS, J. W. J. Algorithm 232: Heapsort. Communications of the ACM, v. 7, n. 6, p. 347–348, 1964.